UNIVERSIDAD DE ANTIOQUIA

Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

Arquitectura de Software

Laboratorio 2

Entorno de desarrollo colaborativo con Spring Boot, GitHub Actions, SonarCloud, Docker y despliegue en Render

Sara Mesa Gómez

Profesor:
Diego José Luis Botia

13 de octubre de 2026

1. Introducción

La integración continua y el despliegue continuo (CI/CD) son prácticas que automatizan la construcción, las pruebas, el análisis de calidad y la entrega de una aplicación cada vez que el código cambia. Su propósito es detectar errores lo antes posible, reducir los conflictos al integrar el trabajo de varias personas y lograr que el software esté siempre en condiciones de ser desplegado.

En este laboratorio se tomó como base el proyecto desarrollado en el Laboratorio 1: una aplicación bancaria para gestionar clientes y realizar transferencias entre cuentas, con persistencia en MySQL y un frontend web servido por la misma aplicación. Sobre este proyecto se construyó un pipeline en GitHub Actions que ejecuta pruebas automatizadas, analiza la calidad del código con SonarCloud y JaCoCo, empaqueta el artefacto JAR, genera una imagen Docker publicada en Docker Hub y despliega la aplicación en la plataforma en la nube Render, conectada a una base de datos MySQL administrada en Aiven.

El trabajo se organizó siguiendo la estrategia de desarrollo trunk-based: cada etapa del pipeline se desarrolló en una rama de corta duración, se integró a la rama principal mediante un Pull Request validado por el propio pipeline y solo los cambios integrados en main llegan a producción. Este informe describe las herramientas utilizadas, la arquitectura propuesta, el procedimiento seguido con sus evidencias y las conclusiones obtenidas.

2. Objetivos

Objetivo general
Implementar un entorno de desarrollo colaborativo con integración y despliegue continuos para la aplicación Spring Boot del Laboratorio 1, utilizando GitHub, GitHub Actions, SonarCloud, Docker y un proveedor en la nube, de modo que cada cambio aprobado en la rama principal se pruebe, se analice y se despliegue automáticamente.

Objetivos específicos

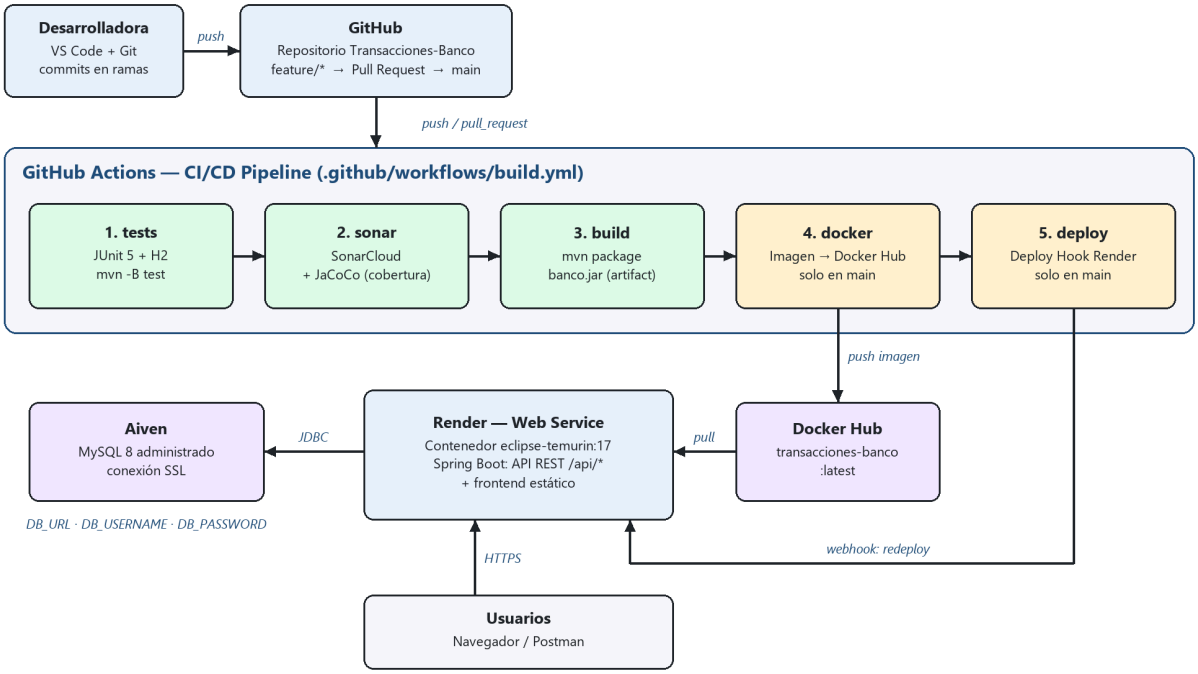
- Configurar un pipeline en GitHub Actions que ejecute automáticamente las pruebas unitarias del proyecto en cada push y Pull Request, usando una base de datos en memoria (H2) independiente del entorno local.
- Integrar SonarCloud y JaCoCo al pipeline para evaluar la calidad del código, los problemas de mantenibilidad, los puntos críticos de seguridad y la cobertura de pruebas.
- Automatizar la construcción del artefacto JAR y su empaquetado en una imagen Docker publicada en Docker Hub.
- Desplegar la aplicación en Render conectado a una base de datos MySQL, configurando las credenciales mediante variables de entorno y secretos.
- Aplicar la estrategia trunk-based mediante ramas de corta duración, Pull Requests y merges a la rama principal, configurando las ramas en el archivo build.yml.
- Verificar el funcionamiento del backend desplegado en la nube mediante pruebas de los servicios REST con Postman.

3. Herramientas de software empleadas

Herramienta	Uso en el laboratorio
Java 17 y Spring Boot 3.5	Lenguaje y framework de la aplicación (Spring Web, Spring Data JPA, Validation). Expone la API REST y sirve el frontend estático.
Maven (Maven Wrapper)	Gestión de dependencias y ciclo de construcción: compilación, pruebas (mvn test) y empaquetado del JAR (mvn package).
MySQL 8	Base de datos relacional de la aplicación, tanto en el entorno local como en la nube.
H2 Database	Base de datos en memoria utilizada únicamente durante las pruebas, para que el pipeline no dependa de un MySQL instalado.
JUnit 5 y Spring Boot Test	Framework de pruebas automatizadas que ejecuta el pipeline.
JaCoCo 0.8.12	Plugin de Maven que mide la cobertura de código de las pruebas y genera el reporte que consume SonarCloud.
Git y GitHub	Control de versiones y repositorio remoto. Gestión de ramas, Pull Requests y secretos del repositorio.
GitHub Actions	Plataforma de CI/CD donde se ejecuta el pipeline definido en .github/workflows/build.yml.
SonarCloud (SonarQube Cloud)	Análisis estático de calidad: bugs, code smells, vulnerabilidades, security hotspots, duplicación y Quality Gate.
Docker y Docker Hub	Empaquetado de la aplicación en una imagen de contenedor (eclipse-temurin:17) y registro público de la imagen.
Render	Proveedor en la nube (PaaS) que ejecuta el contenedor como Web Service con URL pública y HTTPS. Se redespliega mediante un Deploy Hook.
Aiven for MySQL	Base de datos MySQL administrada en la nube, con conexión cifrada por SSL, usada por la aplicación desplegada.
Postman	Cliente HTTP utilizado para probar los servicios REST desplegados en la nube.
Visual Studio Code	Editor de código y terminal integrado para el desarrollo y la ejecución local

4. Arquitectura propuesta

La arquitectura propuesta tiene dos partes que trabajan juntas: el flujo de integración y despliegue continuo, que lleva el código desde el repositorio hasta producción, y la infraestructura de ejecución, donde la aplicación atiende a los usuarios.



La aplicación es un monolito Spring Boot organizado por capas. Los controladores REST (CustomerController y TransactionController) reciben las peticiones HTTP en las rutas /api/customers y /api/transactions; la capa de servicios contiene la lógica de negocio, como validar el saldo antes de una transferencia; los repositorios de Spring Data JPA acceden a la base de datos; y los DTO junto con los mappers de MapStruct separan el modelo expuesto en la API de las entidades persistidas. El frontend (HTML, CSS y JavaScript) es contenido estático desde la misma aplicación, por lo que un único contenedor publica la interfaz web y la API.

Para que funcione en cualquier entorno, la configuración sensible se externalizó en variables de entorno con valores por defecto para el desarrollo local: PORT, DB_URL, DB_USERNAME y DB_PASSWORD..

Pipeline de CI/CD

El pipeline está definido en el archivo .github/workflows/build.yml y se compone de cinco trabajos (jobs) encadenados con needs, de modo que cada uno solo se ejecuta si el anterior terminó con éxito:

Job	Cuándo se ejecuta	Qué hace
tests	Push y Pull Request	Ejecuta mvn -B test con JDK 17 sobre una base H2 en memoria.
sonar	Push y Pull Request	Ejecuta mvn verify con JaCoCo y envía el análisis a SonarCloud, que evalúa el Quality Gate.
build	Push y Pull Request	Empaqueta la aplicación (mvn package) y publica banco.jar como artefacto del workflow.
docker	Solo en main	Descarga el JAR, construye la imagen con el Dockerfile y la publica en Docker Hub como transacciones-banco:latest.
deploy	Solo en main	Invoca el Deploy Hook de Render, que descarga la nueva imagen y redespliega el servicio.

Las credenciales nunca se escriben en el código. Se almacenan como secretos del repositorio en GitHub (SONAR_TOKEN, DOCKER_USERNAME, DOCKER_PASSWORD y RENDER_DEPLOY_HOOK) y en las variables de entorno del servicio en Render.

Estrategia de ramas trunk-based

La rama main actúa como tronco y siempre debe estar en un estado desplegable. Cada etapa del laboratorio se desarrolló en una rama de corta duración (feature/ci-tests, feature/sonar-jacoco, feature/docker y feature/render-deploy). En la rama y en el Pull Request se ejecutan las validaciones (tests, sonar y build); solo después del merge a main se ejecutan los jobs docker y deploy.

Infraestructura de ejecución

En producción, Render ejecuta la imagen publicada en Docker Hub como un Web Service con HTTPS en la URL pública de la aplicación. Render asigna el puerto mediante la variable PORT, que la aplicación lee al iniciar. La base de datos es un servicio MySQL 8 administrado en Aiven, al que la aplicación se conecta por JDBC con SSL obligatorio. Los usuarios acceden desde el navegador al frontend o, como en las pruebas de este laboratorio, directamente a la API REST mediante Postman.

5. Procedimiento

El pipeline se construyó de forma incremental: cada etapa se desarrolló en su propia rama, se validó con el pipeline en un Pull Request y se integró a main antes de pasar a la siguiente.

Etapa 1: pruebas automatizadas (feature/ci-tests)
El único test del proyecto (contextLoads) levanta el contexto completo de Spring, que intenta conectarse a MySQL en localhost, se agregó la dependencia de H2 con alcance test y un archivo src/test/resources/application.properties que reemplaza la configuración durante las pruebas:

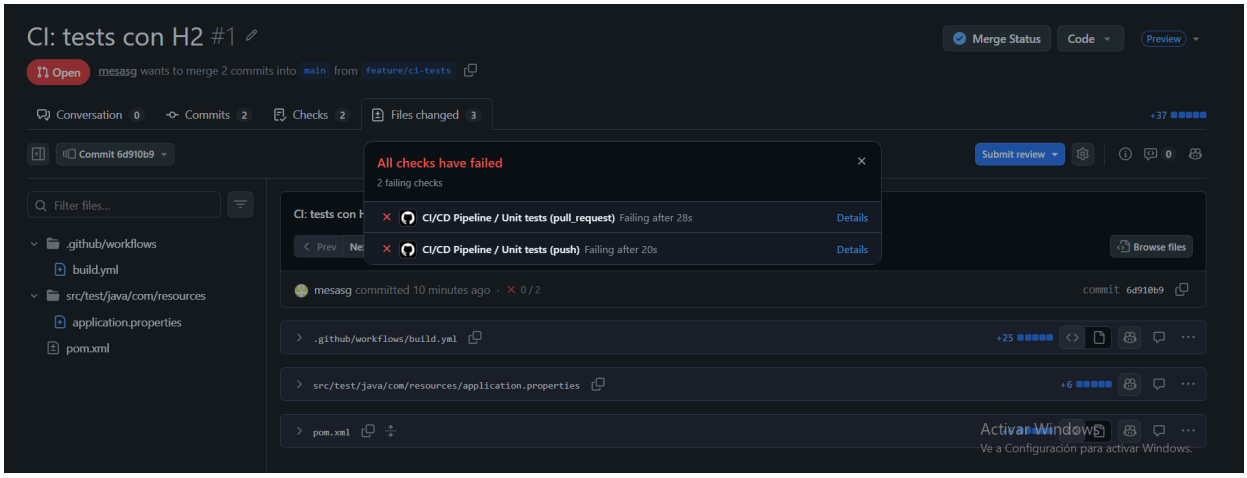
```
spring.datasource.url=jdbc:h2:mem:testdb;MODE=MySQL;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
```

```
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
```

Luego se creó el archivo .github/workflows/build.yml con el job tests y se verificó que las pruebas pasaran localmente.



Al abrir el Pull Request #1, el pipeline falló aunque las pruebas pasaban en el equipo local.. El log mostró que Maven no encontraba src/test/resources: el archivo de configuración de pruebas había quedado en src/test/java/com/resources, así que el test seguía usando MySQL. En el equipo local no se notaba porque había un MySQL corriendo. Este resultado muestra el valor de la integración continua: el pipeline detectó un error que dependía del entorno y que localmente pasaba inadvertido.



Se movió el archivo a src/test/resources el pipeline pasó y se hizo el merge del Pull Request #1 a main.

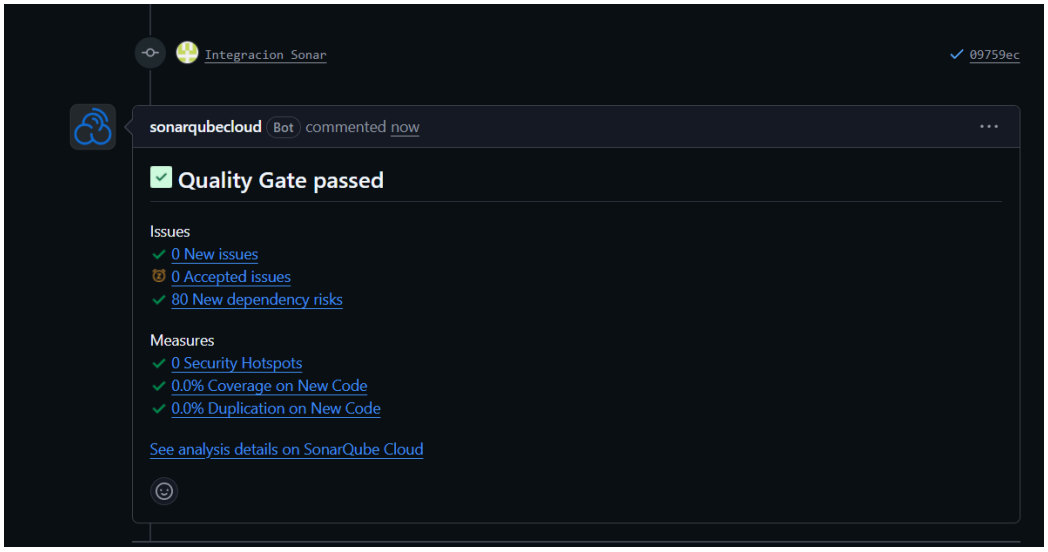
Etapa 2: Análisis de calidad con SonarCloud y JaCoCo (feature/sonar-jacoco)

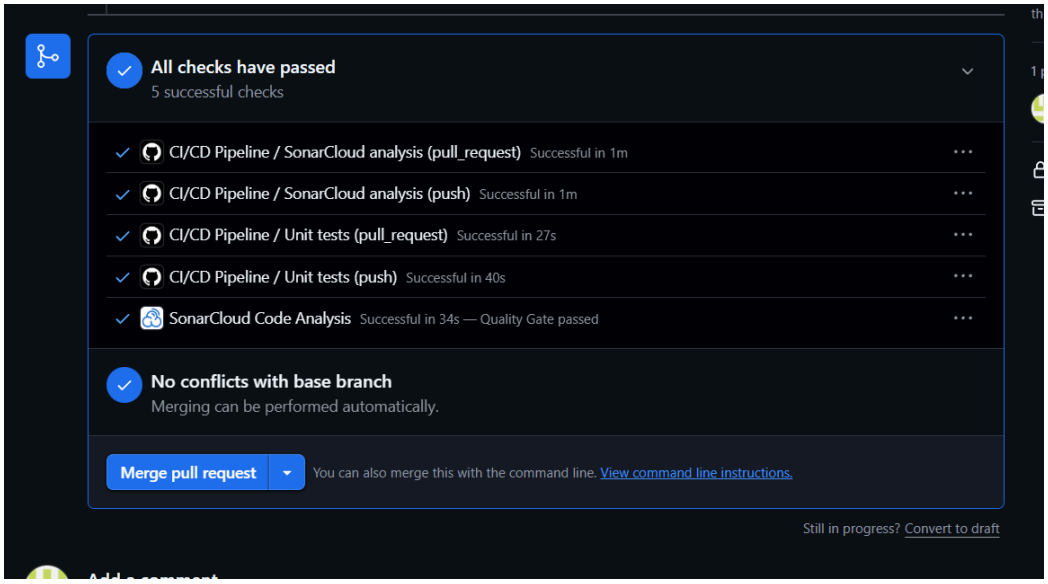
En SonarCloud se desactivó el Automatic Analysis, porque el análisis lo realiza el pipeline. Se generó un token y se registró en GitHub como el secreto SONAR_TOKEN.

En el pom.xml se agregaron las propiedades del proyecto en SonarCloud y el plugin de JaCoCo, que genera el reporte de cobertura durante la fase de pruebas.

```
<sonar.projectKey>mesasg_Transacciones-Banco</sonar.projectKey>
<sonar.organization>mesasg</sonar.organization>
<sonar.host.url>https://sonarcloud.io</sonar.host.url>
```

En build.yml se agregó el job sonar, dependiente de tests, que ejecuta mvn -B verify y el goal sonar del scanner de Maven. En el Pull Request #2 SonarCloud publicó su resultado: Quality Gate aprobado y los 5 checks en verde.





Etapa 3: construcción del JAR e imagen Docker (feature/docker)

Se fijó el nombre del artefacto con <finalName>banco</finalName> en el pom.xml y se externalizó la configuración en src/main/resources/application.properties mediante variables de entorno con valores por defecto para el entorno local:

```
server.port=${PORT:8080}
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/lab12026p?useSSL=false&...}
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD:root}
```

Se creó un Dockerfile en la raíz del proyecto que toma el JAR construido por el pipeline y lo ejecuta sobre una imagen ligera de Java 17.

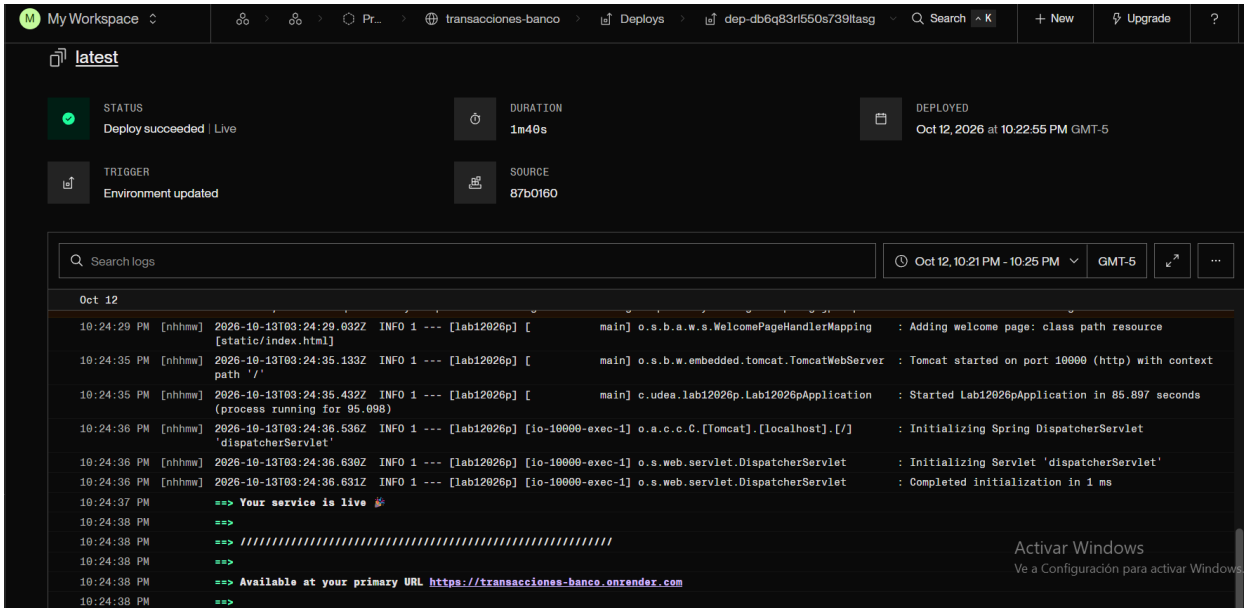
En build.yml se agregaron dos jobs. El job build empaqueta la aplicación y publica banco.jar como artefacto del workflow. El job docker, descarga el artefacto, inicia sesión en Docker Hub y construye y publica la imagen transacciones-banco:latest. Las credenciales se registraron como los secretos DOCKER_USERNAME y DOCKER_PASSWORD que usa un Personal Access Token de Docker Hub en lugar de la contraseña.

Etapa 4: Despliegue en la nube con Render (feature/render-deploy)

La guía del laboratorio usa una aplicación de ejemplo sin base de datos, pero el proyecto del Laboratorio 1 persiste la información en MySQL. Por eso, se usó Aiven. Los pasos fueron:

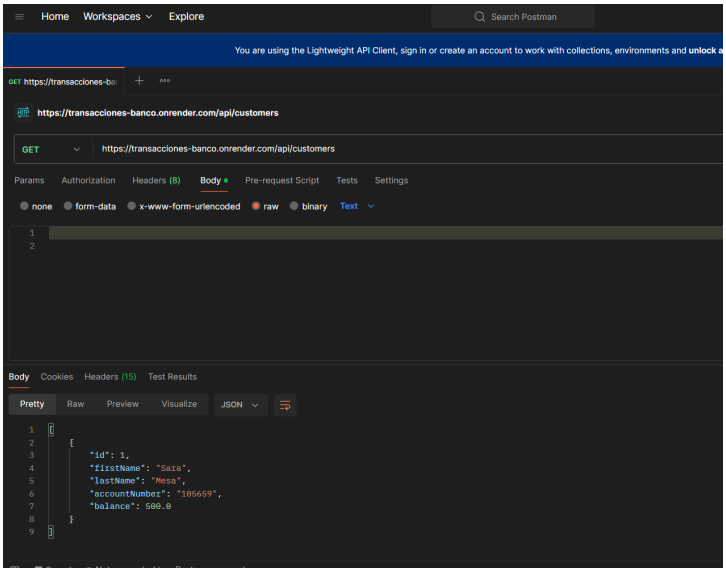
1. Base de datos en Aiven. Se creó un servicio MySQL con el plan gratuito. Como la entidad Customer usa GenerationType.AUTO, Hibernate crea una tabla de secuencia sin llave primaria; por eso se desactivó el parámetro sql_require_primary_key en la configuración avanzada del servicio.
2. Web Service en Render. Se creó el servicio a partir de la imagen existente docker.io/<usuario>/transacciones-banco:latest con el plan gratuito, y se configuraron las variables de entorno DB_URL, DB_USERNAME y DB_PASSWORD. La variable PORT la define Render automáticamente.
3. Formato de la conexión. La URI que entrega Aiven no está en formato JDBC; la variable DB_URL quedó con la forma jdbc:mysql://HOST:PUERTO/defaultdb?sslMode=REQUIRED, con el usuario y la contraseña en sus propias variables.
4. Integración con el pipeline. Se copió el Deploy Hook del servicio (Settings → Deploy Hook) y se registró en GitHub como el secreto RENDER_DEPLOY_HOOK. En build.yml se agregó el job deploy, dependiente de docker, que invoca el hook con curl para que Render descargue la nueva imagen y redespliegue el servicio.

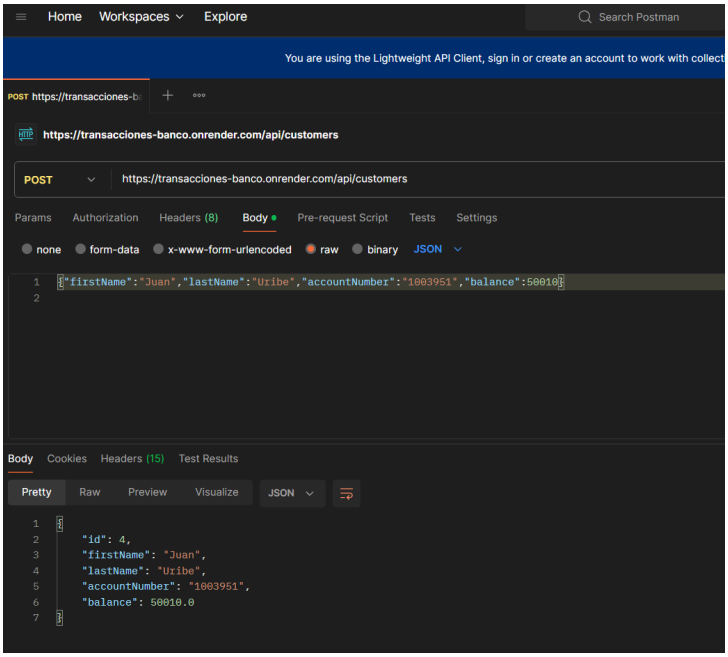
El despliegue terminó con éxito y el servicio quedó publicado,

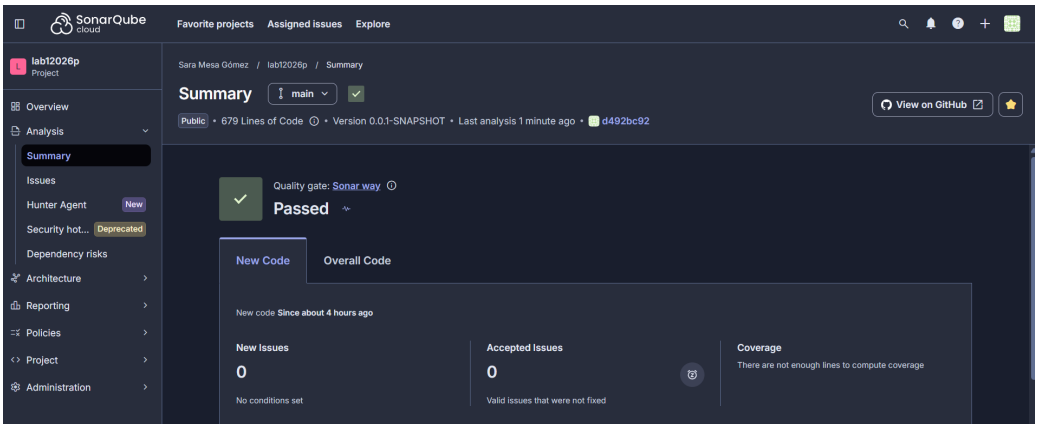


Con la aplicación desplegada en https://transacciones-banco.onrender.com, se probaron los servicios REST con Postman. Los servicios disponibles son:

Método	Endpoint	Descripción
GET	/api/customers	Lista todos los clientes.
GET	/api/customers/{id}	Consulta un cliente por su id.
POST	/api/customers	Crea un cliente con nombre, apellido, número de cuenta y saldo.
PUT	/api/customers/{id}	Actualiza un cliente.
DELETE	/api/customers/{id}	Elimina un cliente.
POST	/api/transactions	Transfiere dinero entre dos cuentas, validando el saldo disponible.
GET	/api/transactions/{accountNumber}	Consulta el historial de transacciones de una cuenta.


Home
Workspaces
Explore
Search Postman
You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock a
GET https://transacciones-ba
https://transacciones-banco.onrender.com/api/customers
GET
https://transacciones-banco.onrender.com/api/customers
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
none
form-data
x-www-form-urlencoded
raw
binary
Text
Body
Cookies
Headers (15)
Test Results
Pretty
Raw
Preview
Visualize
JSON
{
"id": 1,
"firstName": "Sara",
"lastName": "Mesa",
"accountNumber": "105659",
"balance": 500.0
}


Home
Workspaces
Explore
Search Postman
You are using the Lightweight API Client, sign in or create an account to work with collect
POST https://transacciones-b
https://transacciones-banco.onrender.com/api/customers
POST
https://transacciones-banco.onrender.com/api/customers
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
none
form-data
x-www-form-urlencoded
raw
binary
JSON
{"firstName":"Juan","lastName":"Uribe","accountNumber":"1003951","balance":50010}
Body
Cookies
Headers (15)
Test Results
Pretty
Raw
Preview
Visualize
JSON
{
"id": 4,
"firstName": "Juan",
"lastName": "Uribe",
"accountNumber": "1003951",
"balance": 50010.0
}


SonarQube cloud
Favorite projects
Assigned issues
Explore
lab12026p
Project
Overview
Analysis
Summary
Issues
Hunter Agent
New
Security hot...
Deprecated
Dependency risks
Architecture
Reporting
Policies
Project
Administration
Sara Mesa Gómez / lab12026p / Summary
Summary
main
Public • 679 Lines of Code • Version 0.0.1-SNAPSHOT • Last analysis 1 minute ago • d492bc92
View on GitHub
Quality gate: Sonar way
Passed
New Code
Overall Code
New code Since about 4 hours ago
New Issues
0
No conditions set
Accepted Issues
0
Valid issues that were not fixed
Coverage
There are not enough lines to compute coverage

6. Conclusiones

- La integración continua detecta errores que localmente pasan inadvertidos. En la etapa 1 las pruebas pasaban en el equipo por el MySQL local, pero fallaron en el pipeline por la ubicación incorrecta del archivo de configuración. Ejecutar las pruebas en un entorno limpio y reproducible expuso el problema antes de integrarlo a main.
- Aislar las pruebas de la infraestructura las hace confiables. Usar H2 en memoria durante las pruebas permitió que el pipeline se ejecute en cualquier runner sin depender de servicios externos.
- Externalizar la configuración permite desplegar un único artefacto en varios entornos. Gracias a las variables de entorno, la misma imagen Docker funciona en local y en la nube, y las credenciales permanecen fuera del código, protegidas como secretos de GitHub y variables de Render.
- El desarrollo trunk-based con Pull Requests protege la rama principal. Cada etapa se integró solo después de pasar las pruebas y el Quality Gate, y el despliegue se restringió a main. Además, el historial de ramas y Pull Requests documenta la evolución del pipeline.
- El análisis estático complementa las pruebas, pero la cobertura sigue siendo baja. SonarCloud aprobó el Quality Gate sin issues ni security hotspots. Sin embargo, el proyecto solo cuenta con la prueba de carga del contexto, así que la cobertura es baja.
- Los logs son la principal herramienta de diagnóstico en la nube. Los errores que se presentaron en el desarrollo se identificaron y resolvieron leyendo los mensajes de GitHub Actions y Render, lo que muestra la importancia de la observabilidad en un pipeline automatizado.

7. Bibliografía

mesasg. (2026). Transacciones-Banco [Repositorio de código]. GitHub. https://github.com/mesasg/Transacciones-Banco

Botia, D. J. L. (2026). Laboratorio: Implementación de un entorno colaborativo de integración continua con Spring Boot, GitHub Actions y SonarCloud [Guía de laboratorio]. Departamento de Ingeniería de Sistemas, Universidad de Antioquia.

URL´s

Recurso	Enlace
Repositorio	https://github.com/mesasg/Transacciones-Banco
Aplicación desplegada	https://transacciones-banco.onrender.com
Pipeline (GitHub Actions)	https://github.com/mesasg/Transacciones-Banco/actions
Análisis en SonarCloud	sonarcloud.io — mesasg_Transacciones-Banco